JAVA SDE-2 INTERVIEW PREPARATION SERIES

DATA STRUCTURES AND ALGORITHMS - BOOK 09 OF 17 - STUDY STEP DSA-09

Recursion and Backtracking in Java

Contracts, Call Stacks, Search Trees, Pruning, and State Restoration

BY

Vinay Reddy Kalluri

Reason from recursive contracts, understand stack cost, restore mutable state correctly, and prune combinatorial search without losing solutions.

RECURSION | STACK FRAMES | BACKTRACKING | PRUNING | SEARCH TREES

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `DSA 08 - Hashing and Prefix State`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

This volume separates recursion as a method contract from backtracking as controlled state-space search, with Java stack and mutation behavior made explicit.

Readiness map

Decision	Evidence
START HERE	The problem decomposes into smaller instances of the same contract; A decision tree must enumerate valid combinations; State must be chosen, explored, and undone; Input depth can threaten the Java stack.
PREPARE FIRST	JAVA-01 and DSA-01 through DSA-08; Complexity and collection fundamentals.
MOVE ON WHEN	Write precise recursive contracts and base cases; Analyze call depth and output-sensitive work; Implement choose-explore-unchoose safely; Apply pruning without invalidating completeness.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

DSA Segment Position

DSA 08 - Hashing and Prefix State	**YOU ARE HERE** **DSA 09 - Recursion and Backtracking**	DSA 10 - Linked Lists

A note from Vinay

Recursion is easiest to trust when one call has a precise promise. Write that promise, prove the base case, and show how the recursive step reduces the remaining work.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Recursion Foundations: Contracts Before Search Trees

Recursion is not "a method calling itself until it stops." A reliable recursive method states what one call promises, handles the smallest complete cases, and reduces every other case toward them.

Translate a loop into a smaller problem

To sum the first `n` array elements, define this contract:

TEXT

```
sum(values, n) returns values[0] + ... + values[n - 1]
```

The smallest complete instance is `n == 0`, whose sum is zero. For `n > 0`, the answer is the sum of the first `n - 1` elements plus `values[n - 1]`.

JAVA

```java
static long sum(int[] values, int n) {
    if (n == 0) {
        return 0L;
    }
    return sum(values, n - 1) + values[n - 1];
}
```

For `[4, 7, 2]`:

TEXT

```
sum(a, 3)
  waits for sum(a, 2) + 2
    waits for sum(a, 1) + 7
      waits for sum(a, 0) + 4
        returns 0
      returns 4
    returns 11
  returns 13
```

Each active call owns its parameter values, local variables, and return location. Java does not guarantee tail-call optimization, so recursion depth is real auxiliary stack usage.

The four correctness questions

Before coding, answer:

1. **Contract:** What exactly does one call return or accomplish?
2. **Base case:** Which smallest inputs can be answered directly?
3. **Progress:** Why does every recursive branch move toward a base case?
4. **Combination:** How do smaller answers form the current answer?

A missing base case can cause `StackOverflowError`. A base case that is never reached is equally broken. A recursive call on the same state does not make progress.

Return-value recursion versus mutable-state recursion

Return-value recursion computes an answer from child answers, as in height or sum. Mutable-state recursion updates shared or caller-owned state, as in building a list of paths. State ownership must be explicit.

JAVA

```java
static void collectCountdown(int n, List<Integer> output) {
    if (n == 0) {
        return;
    }
    output.add(n);
    collectCountdown(n - 1, output);
}
```

The reference value is passed by value, but both caller and callee refer to the same list object. Mutations remain visible after the call returns.

From recursion to a decision tree

For subsets, each index offers two choices: exclude the value or include it.

TEXT

```
                        [] at index 0
                   /                    \
             exclude 1                  include 1
               []                         [1]
           /        \                 /        \
      exclude 2   include 2       exclude 2   include 2
         []          [2]             [1]        [1,2]
```

The path is the partial candidate. The index is the next decision. A leaf is a complete set of decisions.

Backtracking is reversible mutation

The standard transaction is:

```text
choose -> explore -> unchoose
```

```java
static void subsets(int[] values, int index,
                    List<Integer> path,
                    List<List<Integer>> output) {
    if (index == values.length) {
        output.add(new ArrayList<>(path));
        return;
    }

    subsets(values, index + 1, path, output);

    path.add(values[index]);
    subsets(values, index + 1, path, output);
    path.remove(path.size() - 1);
}
```

The copy at the leaf is essential. Adding `path` directly would store repeated references to one mutable list. The removal is equally essential: on return, the caller must see the path exactly as it was before the choice.

State ownership options

State	Typical owner	Restoration rule
`index`, remaining target	current call	automatic; values are local
`path` list	shared along one DFS path	undo the last choice
`used[]` for permutations	shared constraint state	reset the chosen position
board marker	shared input or work buffer	restore before return
output list	whole search	append completed snapshots; do not undo output

Use `try/finally` when restoration must occur even if deeper code can throw:

```java
char saved = board[row][column];
board[row][column] = '#';
try {
    return searchNeighbor(board, row, column);
} finally {
    board[row][column] = saved;
}
```

Pruning must be proved safe

Pruning removes branches that cannot produce a valid or better answer. Examples:

- stop a fixed-size combination when too few elements remain;
- stop a positive-number combination when the remaining target is negative;
- skip a value already used at the same depth when generating unique permutations;
- stop optimization search when an admissible lower bound is no better than the best result.

Do not copy a pruning rule across contracts. `remaining < 0` is unsafe when later values can be negative.

Complexity without hand-waving

Count three separate quantities:

- number of decision-tree nodes visited;
- work performed per node, including copying a path;
- maximum call depth and mutable state size.

Generating all subsets has `2^n` leaves and must output `n * 2^(n-1)` element occurrences across all subsets. The algorithm is output-sensitive; saying only O(2^n) hides copying cost.

When not to recurse

- Depth can be proportional to a large untrusted input.
- An iterative loop expresses a linear scan more clearly.
- A graph traversal may exceed the Java stack; an explicit deque gives controlled heap-backed storage.
- Repeated subproblems indicate memoization or dynamic programming, not plain backtracking.

TRY IT | Foundation checkpoint

1. State the contract and base case for recursive binary-tree height.
2. Why must `path` be copied at a solution leaf?
3. What state is restored in a permutation search?
4. Why is recursion depth part of auxiliary space?
5. What proof is required before adding a prune?

SDE-2 CORE CHAPTER

Chapter 2 - Recursive Contracts and Backtracking Patterns for SDE-2

Recursion is not a trick for making a loop disappear. It is a way to delegate a smaller instance of a problem under a precise contract. Backtracking adds a second idea: make one reversible decision, explore everything below it, then restore the caller's state. At SDE-2 level, an answer is incomplete if it merely produces the right combinations. You should be able to name the state, prove that every legal answer is generated exactly as intended, bound the search tree, and explain who owns each mutable object.

The pattern map

Signal in the prompt	State to carry	Typical pattern
"Generate every subset"	next index and current selection	include/exclude or start-index DFS
"Choose k from n"	next candidate, choices remaining	combinations with a feasibility bound
"Use values to reach a target"	start index and remainder	combination search with numeric pruning
"Every ordering"	used positions and current order	permutation search
"Find a path spelling a word"	board coordinate and word offset	grid DFS with temporary visitation
"Place items without conflicts"	row and occupied constraints	constraint backtracking, often bit/set state
"Count/optimize, answers overlap"	arguments that fully identify a subproblem	memoization or dynamic programming

The recognition distinction matters. A traversal follows edges already present in the input. Backtracking constructs a decision tree that is usually not stored anywhere. Memoization caches the result of a state only when that result is independent of the path used to reach it.

Start with a recursive contract

For every helper, say one complete sentence before coding:

> `search(i, path)` appends every valid completion whose decisions before `i` are exactly `path`, and leaves `path` unchanged when it returns.

That sentence gives four proof obligations.

1 **Meaning:** the parameters describe all information needed below this call.

2. **Base case:** a smallest state returns the correct result without another call.
3. **Progress:** every recursive edge moves toward a base case.
4. **Preservation:** mutable state observed by the caller is restored before return.

A recursion proof is structural induction. Assume calls on smaller states satisfy the contract. Show that the current call partitions its legal answers into the recursive branches, combines their results correctly, and terminates. The code should mirror that proof.

Stack and allocation reality in Java

Each ordinary Java call consumes a stack frame. Java does not guarantee tail-call elimination, so a logically tail-recursive method can still overflow the stack. Depth `O(log n)` is normally comfortable; depth proportional to an untrusted input may not be. A million-node chain, a long flood fill, or a degenerate tree should trigger an iterative design or an explicit stack.

Allocation is separate from recursion depth. A `new ArrayList<>(path)` at every leaf is required when returning snapshots; returning the live `path` would alias one object and corrupt all results. Copying at every internal node, however, can add unnecessary allocation. Prefer one owned mutable path, mutate/recurse/undo, and copy only when publishing an answer.

Backtracking as a transaction

The invariant is:

> On entry to a call, shared mutable state represents exactly the decisions on the path from the root to this node; on exit, it is byte-for-byte or logically equivalent to its entry state.

Use a transaction-shaped sequence:

1. choose;
2. mutate the owned search state;
3. recurse;
4. undo in the reverse order.

If the recursive call can throw and the state must remain reusable, production code may need `try/finally` around the undo. Interview problems normally assume no exception inside the search, but naming the issue demonstrates ownership awareness.

Family 1: subsets

Recognition and invariant

Use subset DFS when each element may be selected at most once and output order does not matter. With a start-index formulation, `start` is the first undecided position. `path` contains an increasing sequence of positions, so no subset is generated twice. At every call, the current path is itself a valid subset and can be emitted.

TRACE IT | Dry run

For `[1, 2]`, emit `[]`. Choose index `0`: emit `[1]`; choose index `1`: emit `[1,2]`; undo `2`, undo `1`. Then choose index `1`: emit `[2]`. The answer sequence is `[], [1], [1,2], [2]`. More important than this order is the restoration: after the child for `[1,2]` returns, the caller again owns `[1]`.

Correctness and cost

Every subset has one unique increasing list of indexes. The loop follows exactly that list, so it reaches every subset once. There are `2^n` answers and copying each can cost `O(n)`, giving `O(n * 2^n)` output time and `O(n)` search depth excluding output. No algorithm can return all subsets in sub-output-size time.

Edges and mistakes

The empty input has one subset: the empty subset. If equal input values should not create duplicate value-subsets, sort first and skip equal candidates at the same depth; the plain implementation treats positions as distinct. Do not append `path` itself to results. Append a snapshot.

Family 2: fixed-size combinations and combination sum

Recognition and pruning

Combinations choose without regard to order. In `choose k from 1..n`, carry the next legal number. A useful bound is: if `need` values remain, the largest legal starting value is `n - need + 1`. Iterating past it creates calls that cannot possibly finish.

For combination sum with reusable positive candidates, sort and carry a `start` index. Reusing a candidate calls the child with the same index; disallowing reuse calls with `i + 1`. Positivity makes `candidate > remainder` a sound stopping rule. That prune is invalid if negative or zero candidates are allowed, so the API contract must reject them or use a different state model.

Invariant and dry run

For candidates `[2,3,6,7]` and target `7`, the path is nondecreasing because future choices begin at `start`. From `[]`, choose `2`, then `2`, then `2`; remainder `1` prunes all choices. Restore to `[2,2]`, then try `3`, reaching remainder `0` and publishing `[2,2,3]`. Later the root chooses `7` and publishes `[7]`. Ordering plus same-depth duplicate skipping prevents permutation duplicates such as `[2,3,2]`.

The invariant is that `sum(path) + remainder` equals the original target, every element in `path` came from the candidate set, and path indexes are nondecreasing. A remainder of zero is therefore a valid answer. With positive candidates, every recursive call reduces the remainder, proving termination.

Complexity

Combination generation costs at least the total output size. A coarse search bound for choose-`k` is `O(C(n,k) * k)`. Combination sum can have exponentially many nodes; with minimum value `m`, depth is at most `target / m`. State the output-sensitive reality instead of claiming a misleading single polynomial bound.

Family 3: unique permutations

Recognition and invariant

Permutations care about order, so a start index is insufficient. Carry a `used[]` flag for positions. At depth `d`, exactly `d` positions are used and `path` is their ordered sequence. Sort the input. At one depth, skip `values[i]` when it equals `values[i-1]` and the earlier equal position has not been used. This rule chooses one representative among indistinguishable siblings while still allowing equal values at different depths.

For `[1,1,2]`, the root may choose the first `1`, but it skips the second `1` as an equivalent root branch. Below the first `1`, the second can be chosen because its predecessor is already used. The leaves are `[1,1,2]`, `[1,2,1]`, and `[2,1,1]`.

There are at most `n!` leaves and each snapshot costs `O(n)`, so the upper bound is `O(n * n!)` time and `O(n)` auxiliary search state, excluding results. Sorting adds `O(n log n)`.

Family 4: grid Word Search

Recognition and ownership

Use grid backtracking when one path must satisfy an ordered constraint and a cell cannot be reused within that path. The state is `(row, column, offset)` plus visitation. Four neighbors form the branches. A separate visited matrix accepts every `char` value safely. In-place marking is an optional optimization only when a sentinel is provably outside the input alphabet and every return path restores the board.

The helper contract is: starting at this cell with all earlier path cells marked visited, report whether the suffix beginning at `offset` can be matched, and restore the visited state before return. Bounds, prior visitation, and character mismatch reject a state. Matching the last character succeeds.

TRACE IT | Dry run and proof

On board rows `ABCE`, `SFCS`, `ADEE`, searching `ABCCED` starts at `A(0,0)`, advances right to `B`, right to `C`, down to `C`, left to `E`, and left to `D`. Each entered cell is temporarily marked in the visited matrix, so the path cannot turn back and reuse a prior cell. On unwind, every visitation mark is restored.

The branching factor is at most four for the first step and at most three afterward because the previous cell is marked. A safe bound is `O(rows * cols * 4^L)` for word length `L`, with `O(L)` call depth. Useful production prunes include rejecting when `L` exceeds the cell count and starting from the rarer endpoint after comparing board frequencies. Do not silently reverse the caller's word; keep that optimization internal.

Family 5: N-Queens and constraint state

Recognition

N-Queens represents the broader family "place one decision per level while constraints accumulate." Place one queen per row. Then the state needs only occupied columns, descending diagonals `row - col`, and ascending diagonals `row + col`. Sets make the invariant explicit; bit masks are a later optimization when `n` is safely bounded by the integer width.

At row `r`, rows `[0,r)` contain exactly one queen and none attack one another. Trying column `c` is legal precisely when its column and both diagonal keys are absent. Mark all three constraints, recurse to `r+1`, then unmark them. When `r == n`, the invariant proves a complete legal board.

For `n=4`, choosing row positions `[1,3,0,2]` yields `.Q..`, `...Q`, `Q...`, `..Q.`. A branch beginning with column `0` eventually reaches a row with no legal column and backtracks. This failure is information about that branch only; no global "impossible" cache is sound unless the complete constraint state is part of the key.

The worst case is conventionally bounded by `O(n!)` candidate placements, with `O(n)` depth and `O(n)` constraint/path state excluding output. Symmetry breaking can halve root work, but it complicates output enumeration and should be introduced only when the contract permits reconstructing mirrored boards.

Where memoization begins-and where it does not

Memoization is sound when a function's result depends only on its cache key. Counting ways to climb from step `i` depends only on `i`, so cache `i`. A Word Search call that uses `(r,c,offset)` but omits which cells are already visited is not the same subproblem; caching `false` under that incomplete key can reject a valid path. Likewise, N-Queens needs row plus occupied constraints, not row alone.

Ask two questions:

1. If two paths reach the same proposed key, are their legal future choices identical?
2. Is the returned value independent of result-list side effects and caller-owned mutable state?

If either answer is no, expand the key, copy/freeze the relevant state, or do not memoize. Backtracking enumerates path-dependent possibilities; dynamic programming merges truly equivalent states.

Complete Java 21 reference implementation

The class below is intentionally standalone. Every public entry point validates the assumptions on which its pruning depends, and `main` uses assertions as an executable review checklist. Run it with `java -ea RecursionBacktrackingSde2`.

JAVA (continued 1/7)

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

public final class RecursionBacktrackingSde2 {
    private RecursionBacktrackingSde2() {}

    public static List<List<Integer>> subsets(int[] values) {
        if (values == null) throw new IllegalArgumentException("values is null");
        List<List<Integer>> answer = new ArrayList<>();
        subsetsDfs(values, 0, new ArrayList<>(), answer);
        return answer;
    }

    private static void subsetsDfs(int[] a, int start, List<Integer> path,
                                   List<List<Integer>> answer) {
        answer.add(new ArrayList<>(path));
        for (int i = start; i < a.length; i++) {
            path.add(a[i]);
            subsetsDfs(a, i + 1, path, answer);
            path.remove(path.size() - 1);
        }
    }

    public static List<List<Integer>> combine(int n, int k) {
        if (n < 0 || k < 0 || k > n) return List.of();
        List<List<Integer>> answer = new ArrayList<>();
        combineDfs(1, n, k, new ArrayList<>(), answer);
        return answer;
    }
```

JAVA (continued 2/7)

```
    private static void combineDfs(int start, int n, int need,
                                   List<Integer> path,
                                   List<List<Integer>> answer) {
        if (need == 0) {
            answer.add(new ArrayList<>(path));
            return;
        }
        for (int value = start; value <= n - need + 1; value++) {
            path.add(value);
            combineDfs(value + 1, n, need - 1, path, answer);
            path.remove(path.size() - 1);
        }
    }

    public static List<List<Integer>> combinationSum(int[] candidates,
                                                      int target) {
        if (candidates == null || target < 0) {
            throw new IllegalArgumentException("invalid input");
        }
        int[] sorted = candidates.clone();
        Arrays.sort(sorted);
        for (int value : sorted) {
            if (value <= 0) {
                throw new IllegalArgumentException("candidates must be positive");
            }
        }
        List<List<Integer>> answer = new ArrayList<>();
        sumDfs(sorted, 0, target, new ArrayList<>(), answer);
        return answer;
    }

    private static void sumDfs(int[] a, int start, int remaining,
                               List<Integer> path,
```

JAVA (continued 3/7)

```
                            List<List<Integer>> answer) {
        if (remaining == 0) {
            answer.add(new ArrayList<>(path));
            return;
        }
        for (int i = start; i < a.length && a[i] <= remaining; i++) {
            if (i > start && a[i] == a[i - 1]) continue;
            path.add(a[i]);
            sumDfs(a, i, remaining - a[i], path, answer);
            path.remove(path.size() - 1);
        }
    }

    public static List<List<Integer>> permuteUnique(int[] values) {
        if (values == null) throw new IllegalArgumentException("values is null");
        int[] sorted = values.clone();
        Arrays.sort(sorted);
        List<List<Integer>> answer = new ArrayList<>();
        permuteDfs(sorted, new boolean[sorted.length],
                   new ArrayList<>(), answer);
        return answer;
    }

    private static void permuteDfs(int[] a, boolean[] used,
                                   List<Integer> path,
                                   List<List<Integer>> answer) {
        if (path.size() == a.length) {
            answer.add(new ArrayList<>(path));
            return;
        }
        for (int i = 0; i < a.length; i++) {
            if (used[i]) continue;
```

JAVA (continued 4/7)

```
            if (i > 0 && a[i] == a[i - 1] && !used[i - 1]) continue;
            used[i] = true;
            path.add(a[i]);
            permuteDfs(a, used, path, answer);
            path.remove(path.size() - 1);
            used[i] = false;
        }
    }

    public static boolean exists(char[][] board, String word) {
        if (board == null || word == null) {
            throw new IllegalArgumentException("null input");
        }
        if (word.isEmpty()) return true;
        long cells = 0;
        boolean[][] visited = new boolean[board.length][];
        for (int r = 0; r < board.length; r++) {
            char[] row = board[r];
            if (row == null) throw new IllegalArgumentException("null row");
            cells += row.length;
            visited[r] = new boolean[row.length];
        }
        if (word.length() > cells) return false;
        for (int r = 0; r < board.length; r++) {
            for (int c = 0; c < board[r].length; c++) {
                if (wordDfs(board, visited, r, c, word, 0)) return true;
            }
        }
        return false;
    }

    private static boolean wordDfs(char[][] b, boolean[][] visited, int r, int c,
```

JAVA (continued 5/7)

```
                                String word, int offset) {
        if (r < 0 || r >= b.length || c < 0 || c >= b[r].length
                || visited[r][c] || b[r][c] != word.charAt(offset)) return false;
        if (offset == word.length() - 1) return true;
        visited[r][c] = true;
        boolean found = wordDfs(b, visited, r - 1, c, word, offset + 1)
                || wordDfs(b, visited, r + 1, c, word, offset + 1)
                || wordDfs(b, visited, r, c - 1, word, offset + 1)
                || wordDfs(b, visited, r, c + 1, word, offset + 1);
        visited[r][c] = false;
        return found;
    }

    public static List<List<String>> solveNQueens(int n) {
        if (n < 0) throw new IllegalArgumentException("negative n");
        List<List<String>> answer = new ArrayList<>();
        int[] columnAtRow = new int[n];
        boolean[] columns = new boolean[n];
        boolean[] descending = new boolean[Math.max(0, 2 * n - 1)];
        boolean[] ascending = new boolean[Math.max(0, 2 * n - 1)];
        queensDfs(0, n, columnAtRow, columns, descending, ascending, answer);
        return answer;
    }

    private static void queensDfs(int row, int n, int[] positions,
                                  boolean[] columns, boolean[] descending,
                                  boolean[] ascending,
                                  List<List<String>> answer) {
        if (row == n) {
            List<String> board = new ArrayList<>(n);
            for (int r = 0; r < n; r++) {
                char[] line = new char[n];
```

JAVA (continued 6/7)

```
                Arrays.fill(line, '.');
                line[positions[r]] = 'Q';
                board.add(new String(line));
            }
            answer.add(board);
            return;
        }
        for (int col = 0; col < n; col++) {
            int down = row - col + n - 1;
            int up = row + col;
            if (columns[col] || descending[down] || ascending[up]) continue;
            positions[row] = col;
            columns[col] = descending[down] = ascending[up] = true;
            queensDfs(row + 1, n, positions, columns,
                      descending, ascending, answer);
            columns[col] = descending[down] = ascending[up] = false;
        }
    }

    public static long countClimbs(int steps) {
        if (steps < 0) return 0;
        if (steps > 91) throw new ArithmeticException("result exceeds long");
        long[] memo = new long[steps + 1];
        Arrays.fill(memo, -1);
        return countClimbs(steps, memo);
    }

    private static long countClimbs(int remaining, long[] memo) {
        if (remaining == 0) return 1;
        if (remaining < 0) return 0;
        if (memo[remaining] != -1) return memo[remaining];
        return memo[remaining] = Math.addExact(
```

JAVA (continued 7/7)

```
                countClimbs(remaining - 1, memo),
                countClimbs(remaining - 2, memo));
    }

    public static void main(String[] args) {
        assert subsets(new int[] {1, 2, 3}).size() == 8;
        assert combine(4, 2).size() == 6;
        assert combinationSum(new int[] {2, 3, 6, 7}, 7).size() == 2;
        assert permuteUnique(new int[] {1, 1, 2}).size() == 3;

        char[][] board = {
            {'A', 'B', 'C', 'E'},
            {'S', 'F', 'C', 'S'},
            {'A', 'D', 'E', 'E'}
        };
        char[][] before = Arrays.stream(board).map(char[]::clone)
                                .toArray(char[][]::new);
        assert exists(board, "ABCCED");
        assert !exists(board, "ABCB");
        assert !exists(new char[][] {{'A', 'B', 'X'}}, "AB\0");
        assert Arrays.deepEquals(board, before) : "board must be restored";
        assert solveNQueens(4).size() == 2;
        assert countClimbs(5) == 8;
        boolean overflowRejected = false;
        try {
            countClimbs(92);
        } catch (ArithmeticException expected) {
            overflowRejected = true;
        }
        assert overflowRejected;
    }
}
```

Interview-grade edge-case checklist

- Empty choices: subsets returns one empty answer; permutations likewise have one mathematical empty ordering, depending on the API contract.
- Duplicate values: decide whether positions or values define uniqueness. Sort-and-skip only when value uniqueness is required.
- Invalid numeric domains: positivity is part of combination-sum termination and pruning.
- Mutable input: document temporary mutation and restoration when using it. The sample uses call-owned visitation and does not mutate the board, but callers must still keep the input stable while a search is running.
- Ragged grids: horizontal bounds belong to the current row, not row zero. Vertical moves into a different-length row require checking that destination row's width.
- Sentinel collisions: an in-place alternative that marks with `\0` is correct only when that value is outside the input alphabet. The sample's `boolean[][]` removes that assumption at an allocation cost.
- Exponential output: do not promise to materialize an unbounded answer set. A production API might expose an iterator, callback, limit, deadline, or cancellation token.
- Integer overflow: counts can overflow `long` long before the search becomes practically enumerable. The sample rejects climb counts beyond the exact `long` range; another API may use `BigInteger`, saturation, or a documented modulus.
- Stack depth: translate input constraints into maximum depth before choosing recursion.

TRY IT | Exercises with model checkpoints

Exercise 1: duplicate subsets

Return unique subsets of an integer array that may contain duplicates.

Checkpoint: sort a defensive copy; at each depth skip `a[i]` when `i > start && a[i] == a[i-1]`. Do not globally skip duplicates, because two equal values may both appear in one subset. Your invariant should still use increasing positions. Expected complexity is output-sensitive and bounded above by `O(n * 2^n)`.

Exercise 2: combinations without materializing

Expose combinations through `Consumer<List<Integer>>` and stop after a caller-provided limit.

Checkpoint: publish an immutable snapshot or clearly scoped read-only view. Propagate a boolean "stop" result upward so all frames terminate promptly. Validate a nonnegative limit. Discuss whether callbacks may re-enter the generator or throw.

Exercise 3: restore under failure

Modify Word Search so the matching predicate may throw.

Checkpoint: save the character, mark it, and place recursive exploration in `try` with restoration in `finally`. The method's ownership contract then survives both normal and exceptional exits.

Exercise 4: palindrome partitioning

Partition a string into all lists of palindromic substrings.

Checkpoint: the state is a start offset; branches choose an end offset whose slice is a palindrome. Precomputing palindrome truth for all intervals costs `O(n^2)` and avoids rescanning each slice. Copy the path only at `start == length`.

Exercise 5: memoization audit

Explain why `(row, col, offset)` is an insufficient Word Search cache key.

Checkpoint: two calls can share those three values but have different cells unavailable because their earlier paths differ. Their future legal transitions are therefore different. A complete visited-set key is possible but often too large; ordinary backtracking is the appropriate baseline.

Exercise 6: N-Queens count only

Return only the number of solutions using bit masks.

Checkpoint: mask available columns as `all & ~(columns | diagLeft | diagRight)`, extract the lowest set bit, recurse, and shift diagonal attack masks. Define the maximum `n` supported by the chosen primitive and use unsigned shifts where needed. Counting avoids board allocation but not the exponential search.

SDE-2 production follow-ups

How would you make an exponential generator safe in a service? Put explicit limits on input size, results, elapsed time, and memory. Stream results when the transport supports backpressure, propagate cancellation, record truncation in the response contract, and meter the work. Avoid holding locks across callbacks.

Can searches be parallelized? Root branches are often independent after their state is copied. Parallelism is useful only when branches are coarse enough to amortize task and allocation overhead. Shared result ordering, cancellation, and output limits become synchronization concerns. A bounded executor is safer than recursively spawning an unbounded task tree.

Would you mutate the caller's board? In a coding interview, temporary restoration is a standard space optimization. In a library, the default should be a clearly documented ownership contract or a private copy. Concurrent callers, observability hooks, and exceptions make invisible mutation risky.

What evidence would you collect? Count expanded nodes, pruned branches, maximum depth, results produced, and cancellation latency. Those metrics distinguish a poor pruning rule from expensive required output. Benchmark representative distributions, not only one friendly input.

Interview follow-up chain and model answers

Why not pass a fresh path copy into every child? It can be correct, and sometimes it is the clearest ownership choice. However, a depth-`d` path copy at every search node adds allocation and copying beyond the unavoidable snapshots at published leaves. A single caller-owned path with strict restoration usually reduces garbage. I would choose based on branch count, path size, concurrency needs, and whether exceptions/callbacks make restoration fragile.

Can you prove the duplicate-permutation skip? After sorting, equal unused values at the same depth create indistinguishable next paths. The rule permits only the first currently unused representative. It does not suppress the later equal value when the earlier one is already in the path, so `[1,1,2]` remains reachable. Thus equivalent sibling subtrees are collapsed while valid repeated values across depths remain.

What changes when only one answer is required? Return a boolean or optional result upward and short-circuit after the first success. Still restore mutable state before returning. Candidate order then affects which answer is returned and latency, so make ordering deterministic if the API or tests depend on it. Complexity remains exponential in the worst case, but favorable ordering may find a solution early.

When would you replace recursion with an explicit stack? When maximum depth is input-dependent and can exceed the runtime stack budget, when a search must pause/resume, or when checkpointing and cancellation require control over frames. Each explicit frame stores the recursive parameters plus which branch should run next. This moves memory to the heap but does not reduce asymptotic state.

How do you review a proposed prune? Write the constraint as a necessary condition for every completion below the current state. Prove the condition cannot become true again after the branch is discarded. For positive combination sum, `candidate > remainder` is final because later sorted candidates are no smaller. The same statement is false with negative values, so carrying that optimization across contracts is a correctness bug.

How would you count without enumerating? Change the helper return contract from "publish every completion" to "return the number of completions." Then equivalent states may become memoizable, and path snapshots disappear. The count may overflow even when enumeration is impossible; select `long`, `BigInteger`, saturation, or modulo according to the caller's contract. Counting and listing are different APIs with different complexity and memory behavior.

Final review checklist

- I can state the helper contract without referring to its implementation.
- I can identify base case, progress measure, and maximum depth.
- I know whether ordering matters and whether duplicates are positional or value-based.
- Every mutation has a paired restoration before every return.
- Pruning follows from an explicit constraint; it is not a guess.
- Memoization keys include all state that changes future choices.
- Complexity includes both search nodes and the size/cost of returned answers.
- The production API has limits, ownership, cancellation, and overflow policies.

That is the standard for recursion at SDE-2: not merely reaching a leaf, but controlling the contract of the entire search tree.

SDE-2 CORE CHAPTER

Chapter 3 - Essential Recursion and Backtracking Pattern Clinics

Backtracking becomes reliable when every branch is governed by a state contract, a legal-choice rule, and a restoration rule. Balanced-parentheses generation and Sudoku expose those ideas without hiding them behind a large framework.

Clinic 1: generate balanced parentheses

State sentence

`build(open, close)` generates every valid completion after placing `open` opening symbols and `close` closing symbols.

Two constraints remove invalid branches before they exist:

- add `(` only while `open < pairs`;
- add `)` only while `close < open`.

The second rule is the prefix invariant: no prefix of a balanced string has more closing than opening symbols. A result is complete when both counts equal `pairs`.

For three pairs, the first depth-first path chooses all opens and then all closes, producing `(((()))`. Restoration removes the last character before the sibling choice is explored.

Complexity language

There are `C_n` valid results, where `C_n` is the nth Catalan number. Materializing each length-`2n` string costs `O(C_n * n)` output work and output space. Call depth and the mutable builder use `O(n)` auxiliary space, excluding returned strings.

Clinic 2: Sudoku as constraint-state search

Start with validation

The board must be 9 by 9. Every filled character must be a digit from 1 through 9, and the initial rows, columns, and 3-by-3 boxes must contain no duplicates. Rejecting an invalid starting board is different from reporting that a valid starting board has no completion.

State and choice

Boolean tables record whether a digit is already used in each row, column, or box. At an empty cell, a digit is legal only when all three entries are false. Choose it, mark all three entries, recurse, then clear all three and restore the cell if the child fails.

The box index is:

TEXT

```
(row / 3) * 3 + column / 3
```

An SDE-2 optimization chooses the unfilled cell with the fewest legal digits instead of the first empty cell. That minimum-remaining-values heuristic changes search order, not correctness.

Runnable Java 21 clinic

JAVA (continued 1/4)

```java
import java.util.ArrayList;
import java.util.List;

public final class RecursionCoverageClinic {
    private RecursionCoverageClinic() {
    }

    public static List<String> balancedParentheses(int pairs) {
        if (pairs < 0) {
            throw new IllegalArgumentException("pairs must be nonnegative");
        }
        List<String> answer = new ArrayList<>();
        buildParentheses(pairs, 0, 0, new StringBuilder(), answer);
        return answer;
    }

    private static void buildParentheses(int pairs, int open, int close,
            StringBuilder path, List<String> answer) {
        if (open == pairs && close == pairs) {
            answer.add(path.toString());
            return;
        }
        if (open < pairs) {
            path.append('(');
            buildParentheses(pairs, open + 1, close, path, answer);
            path.deleteCharAt(path.length() - 1);
        }
        if (close < open) {
            path.append(')');
            buildParentheses(pairs, open, close + 1, path, answer);
            path.deleteCharAt(path.length() - 1);
        }
```

JAVA (continued 2/4)

```
    }

    public static boolean solveSudoku(char[][] board) {
        validateBoardShape(board);
        boolean[][] rowUsed = new boolean[9][10];
        boolean[][] columnUsed = new boolean[9][10];
        boolean[][] boxUsed = new boolean[9][10];

        for (int row = 0; row < 9; row++) {
            for (int column = 0; column < 9; column++) {
                char cell = board[row][column];
                if (cell == '.') {
                    continue;
                }
                if (cell < '1' || cell > '9') {
                    throw new IllegalArgumentException("invalid cell");
                }
                int digit = cell - '0';
                int box = (row / 3) * 3 + column / 3;
                if (rowUsed[row][digit] || columnUsed[column][digit]
                        || boxUsed[box][digit]) {
                    throw new IllegalArgumentException("duplicate starting digit");
                }
                rowUsed[row][digit] = true;
                columnUsed[column][digit] = true;
                boxUsed[box][digit] = true;
            }
        }
        return solveCell(board, 0, rowUsed, columnUsed, boxUsed);
    }
```

JAVA (continued 3/4)

```
    private static boolean solveCell(char[][] board, int position,
            boolean[][] rowUsed, boolean[][] columnUsed, boolean[][] boxUsed) {
        while (position < 81
                && board[position / 9][position % 9] != '.') {
            position++;
        }
        if (position == 81) {
            return true;
        }

        int row = position / 9;
        int column = position % 9;
        int box = (row / 3) * 3 + column / 3;
        for (int digit = 1; digit <= 9; digit++) {
            if (rowUsed[row][digit] || columnUsed[column][digit]
                    || boxUsed[box][digit]) {
                continue;
            }
            board[row][column] = (char) ('0' + digit);
            rowUsed[row][digit] = true;
            columnUsed[column][digit] = true;
            boxUsed[box][digit] = true;

            if (solveCell(board, position + 1, rowUsed, columnUsed, boxUsed)) {
                return true;
            }

            board[row][column] = '.';
            rowUsed[row][digit] = false;
            columnUsed[column][digit] = false;
            boxUsed[box][digit] = false;
```

JAVA (continued 4/4)

```
        }
        return false;
    }

    private static void validateBoardShape(char[][] board) {
        if (board == null || board.length != 9) {
            throw new IllegalArgumentException("board must have nine rows");
        }
        for (char[] row : board) {
            if (row == null || row.length != 9) {
                throw new IllegalArgumentException("board must be 9 by 9");
            }
        }
    }

    public static void main(String[] args) {
        List<String> values = balancedParentheses(3);
        assert values.size() == 5 && values.contains("(()())");

        char[][] board = {
            "53..7....".toCharArray(), "6..195...".toCharArray(),
            ".98....6.".toCharArray(), "8...6...3".toCharArray(),
            "4..8.3..1".toCharArray(), "7...2...6".toCharArray(),
            ".6....28.".toCharArray(), "...419..5".toCharArray(),
            "....8..79".toCharArray()
        };
        assert solveSudoku(board);
        assert new String(board[0]).equals("534678912");
        System.out.println("PASS essential recursion clinics");
    }
}
```

Expected output with assertions enabled:

TEXT

```
PASS essential recursion clinics
```

Interviewer follow-up chain with model answers

Interviewer: Why not generate all length-`2n` strings and filter them?

Candidate: Filtering visits `2^(2n)` leaves. The prefix invariant prevents every invalid prefix from producing descendants, so the search follows only prefixes that can still complete.

Interviewer: Is the Sudoku solver guaranteed to run quickly?

Candidate: No. Backtracking is exponential in the worst case. Constraint tables make each legality test constant time, and choosing the most constrained cell can reduce the practical tree, but neither changes the worst-case class.

Interviewer: How do you make restoration safe if cancellation can throw?

Candidate: Put the recursive call inside `try` and restore the cell and three constraint entries in `finally`. Then define whether cancellation returns a partial board or guarantees the original board is restored.

SDE-2 CORE CHAPTER

Chapter 4 - Realistic Recursion and Backtracking Interview Rounds

Round 1: subsets with duplicate input values

Prompt and clarification

Return every distinct subset of an integer array that may contain duplicates. The order of returned subsets does not matter.

> May I sort the input, and should I preserve it?

The interviewer permits a sorted copy. That avoids changing caller-owned data and places equal choices together.

Derivation

At each depth, choose the next element from a suffix. Equal values at the same depth create identical subset branches, so skip all but the first. Equal values at different depths are still allowed; `[2, 2]` may be valid.

JAVA

```java
static List<List<Integer>> uniqueSubsets(int[] input) {
    int[] values = input.clone();
    Arrays.sort(values);
    List<List<Integer>> result = new ArrayList<>();
    buildSubsets(values, 0, new ArrayList<>(), result);
    return result;
}

static void buildSubsets(int[] values, int start, List<Integer> path,
                         List<List<Integer>> result) {
    result.add(new ArrayList<>(path));
    for (int i = start; i < values.length; i++) {
        if (i > start && values[i] == values[i - 1]) {
            continue;
        }
        path.add(values[i]);
        buildSubsets(values, i + 1, path, result);
        path.remove(path.size() - 1);
    }
}
```

For `[1, 2, 2]`, the second `2` is skipped only when it competes with the first `2` at the same loop depth. A deeper call may choose it after the first, producing `[2, 2]`.

Follow-up answers

Complexity? Let R be the number of distinct subsets. Copying paths costs $O(\text{total output elements})$, bounded by $O(nR)$. Recursion depth is $O(n)$, excluding output.

Can you avoid sorting? Track values used at each depth with a set, which adds hashing state and usually makes the proof and output order less simple.

Round 2: word search in a board

Prompt

Return whether a word can be formed from horizontally or vertically adjacent cells without reusing a cell in one path.

Strong candidate plan

The state is `(row, column, wordIndex)`. The invariant is that cells already chosen for the current prefix are marked unavailable. A mismatch or boundary ends the branch. A successful match of the last character ends the search.

JAVA

```java
static boolean exists(char[][] board, String word) {
    if (word.isEmpty()) {
        return true;
    }
    for (int row = 0; row < board.length; row++) {
        for (int column = 0; column < board[row].length; column++) {
            if (search(board, word, row, column, 0)) {
                return true;
            }
        }
    }
    return false;
}

static boolean search(char[][] board, String word, int row, int column, int index) {
    if (row < 0 || row >= board.length || column < 0
            || column >= board[row].length || board[row][column] != word.charAt(index)) {
        return false;
    }
    if (index == word.length() - 1) {
        return true;
    }
    char saved = board[row][column];
    board[row][column] = '\0';
    boolean found = search(board, word, row + 1, column, index + 1)
            || search(board, word, row - 1, column, index + 1)
            || search(board, word, row, column + 1, index + 1)
            || search(board, word, row, column - 1, index + 1);
    board[row][column] = saved;
    return found;
}
```

Interviewer follow-ups

What mutation bug appears with early return? Returning immediately after a successful neighbor before restoring the cell leaks the marker into the caller's board. Compute the result, restore, then return, or use `try/finally`.

What about jagged arrays? Bounds must use `board[row].length` for the current row. A rectangular assumption should be stated if required.

Complexity? With `m` cells and word length `L`, a common upper bound is O(m * 3^(L-1)) after the first move because the previous cell cannot be reused, though board shape and repeated characters affect the actual tree. Depth is O(L).

Round 3: palindrome partitioning

Prompt

Partition a string into every sequence of palindromic substrings.

Candidate answer

The decision boundary is the next end index. From `start`, try every substring `[start, end]` that is a palindrome, append it, recurse from `end + 1`, then remove it.

JAVA

```java
static List<List<String>> palindromePartitions(String text) {
    List<List<String>> result = new ArrayList<>();
    partition(text, 0, new ArrayList<>(), result);
    return result;
}

static void partition(String text, int start, List<String> path,
                      List<List<String>> result) {
    if (start == text.length()) {
        result.add(new ArrayList<>(path));
        return;
    }
    for (int end = start; end < text.length(); end++) {
        if (!isPalindrome(text, start, end)) {
            continue;
        }
        path.add(text.substring(start, end + 1));
        partition(text, end + 1, path, result);
        path.remove(path.size() - 1);
    }
}
```

Follow-up answers

Can palindrome checks be improved? Precompute `palindrome[start][end]` in O(n squared) time and space, or memoize checks. Output enumeration can still be exponential.

What changes if only the minimum number of cuts is required? Enumeration is unnecessary. Define a DP state for the minimum cuts from each boundary; that crosses into the Dynamic Programming volume.

What is a useful test set? Empty text, one character, no repeated characters, all identical characters, and a case such as `aab` with both short and long palindrome choices.

Interview closing checklist

State the recursive contract, base case, progress measure, path invariant, restoration rule, safe pruning rule, output cost, maximum depth, and the largest input you would allow on the Java stack.

SDE-2 CORE CHAPTER

Chapter 5 - The Recursion Engine and Backtracking State

Recursion is a way to let each call own one smaller piece of unfinished work. Backtracking adds reversible choices: choose, explore, undo, then try the next choice. If those two ideas are clear, subsets, permutations, combination search, board search, and N-Queens stop looking like unrelated templates.

The complete Java 21 examples are in `RecursionInterviewChecks.java`.

What one call actually owns

For `sum(values, length)`, one call promises: "return the sum of the first `length` values."

TEXT

```
sum([4,7], 2) waits for sum(...,1), then adds 7
  sum([4,7], 1) waits for sum(...,0), then adds 4
    sum([4,7], 0) returns 0
  returns 4
returns 11
```

Each active frame has its own `length` and suspended addition. Local variables are not shared merely because the method is the same. The base case answers the smallest valid problem without another call. The recursive case must move strictly toward it.

The companion records entry and return events so the unwind order is executable, not imaginary.

WHY IT WORKS | Three proof questions

Before writing a recursive method, answer:

1. **Contract:** what does one call return or produce?
2. **Progress:** which measure becomes smaller?
3. **Base:** which smallest state can be answered directly?

For a tree, progress may be moving to a child. For backtracking, it may be increasing the decision depth. "It probably reaches the base" is not a termination argument.

Backtracking is controlled mutation

A standard path search has this lifecycle:

```text
for each legal choice:
    apply choice
    recurse on the smaller remaining decision space
    undo exactly that choice
```

The invariant is:

> On entry at depth `d`, `path` represents exactly the choices made at depths `[0,d)`, and all auxiliary used-state agrees with that path.

Store `new ArrayList<>(path)` when recording an answer. Storing the mutable `path` reference records many aliases to the same object; after undo, every "answer" can appear empty.

Subsets versus permutations

Subsets choose increasing source indexes. After choosing index `i`, recurse from `i + 1`. Order is not part of the result.

Permutations choose any unused index at every depth. A `used[]` array is state, and its mark must be undone after recursion. Order is the result.

For sorted duplicate values, unique permutations use this rule:

```text
skip values[i] when it equals values[i-1]
and the earlier equal index has not been used in this branch
```

That chooses a canonical order among indistinguishable siblings. Skipping every repeated value globally would wrongly prevent using both copies in one permutation.

Reuse versus consume once

In combination sum, recursing with the same candidate index permits reuse. Recursing with `index + 1` consumes a candidate once. This one parameter changes the problem contract.

The companion sorts positive candidates, removes duplicate sibling branches, stops when a candidate exceeds the remaining target, and rejects zero/negative candidates. Without positivity, repeatedly choosing zero never makes progress and negative values invalidate the simple pruning rule.

Board search and state restoration

Word search uses `(row, column, wordIndex)` as logical state and a visited matrix as reversible branch state.

```text
match cell
mark used
explore four neighbors
unmark used before returning
```

The companion does not overwrite board characters with a sentinel, because any chosen sentinel might be valid input. It supports jagged rows by validating bounds against the current row, and it proves the board is unchanged after both successful and failed searches.

Short-circuit `||` is safe only because every explored call restores its own state before returning. Restoration must not be placed after an early `return true` that bypasses cleanup.

N-Queens: prune with maintained constraints

Placing one queen per row removes the need to track used rows. Three boolean arrays track columns, descending diagonals, and ascending diagonals:

```text
column key:              column
descending diagonal:    row - column + n - 1
ascending diagonal:     row + column
```

Choose a safe column, mark all three, recurse to the next row, and unmark. This prunes invalid partial boards immediately. `n=0` has one empty arrangement-the combinatorial identity useful to recursive composition. The teaching implementation caps `n` because solution count and runtime grow rapidly.

Complexity: include the output

Generating `n!` permutations cannot be `O(n)` because the output itself contains `n! * n` values when materialized. State:

- recursion depth;
- branching factor;
- number and size of outputs;
- auxiliary state such as `used[]`; and
- whether copying each answer costs `O(path length)`.

Pruning improves visited state in practice but does not automatically change the worst-case family.

Edge-case matrix

Case	Correct contract	Common failure
empty input	one empty subset/permutation	returning no combinatorial identity
missing base case	reject in review	infinite recursion/stack overflow
huge linear depth	iterative alternative or explicit limit	assuming heap memory prevents stack failure
duplicate values	sort and skip duplicate siblings	duplicate outputs or lost valid copies
mutable path answer	snapshot it	every result aliases the final path
early successful board path	still restore branch state	board/visited state leaks
zero combination candidate	reject under reuse contract	no progress
negative target	reject or define a different bounded problem	invalid `candidate > remaining` pruning
`n=0` queens	one empty arrangement	inconsistent recursion identity
Unicode string search	define code unit/code point model	treating every `char` as a full character

Six live interview Q&A chains

1. Base case and progress

Interviewer: Why does your recursion terminate?

Candidate: One call owns a nonnegative `length`. The recursive branch calls `length - 1`, so the measure strictly decreases, and length zero returns without another call.

Interviewer: What if the caller passes `-1`?

Candidate: That is outside the contract. I validate at the boundary instead of hoping it reaches zero.

2. Result aliasing

Interviewer: Why copy `path` into the result?

Candidate: `path` is reused and undone. Adding the same reference would make all results observe later mutations. A snapshot freezes the answer at that leaf.

Interviewer: What is the cost?

Candidate: `O(path length)` per recorded result, which belongs in output-sensitive complexity.

3. Duplicate permutations

Interviewer: Why is `values[i] == values[i-1]` not enough to skip a duplicate?

Candidate: Two equal copies may both appear in one permutation. I skip the later copy only when its earlier twin is unused at this depth, enforcing sibling order without banning reuse of distinct indexes.

4. Combination candidates

Interviewer: Can your combination-sum code accept zero?

Candidate: Not with unlimited reuse. Choosing zero leaves the remaining target unchanged, so recursion does not progress. I reject nonpositive candidates; a different input contract needs different bounds/state.

5. Board mutation

Interviewer: Why use a visited matrix instead of writing `'#'` into the board?

Candidate: It avoids a sentinel collision and makes preservation explicit. It costs `O(rows*columns)` state. If the domain excludes a sentinel and mutation is permitted, temporary marking can reduce auxiliary space, but restoration is mandatory.

6. Recursion versus iteration

Interviewer: Is recursion always cleaner?

Candidate: It often mirrors a bounded-depth search proof, but a million-node chain can overflow Java's call stack. For untrusted depth I use an explicit stack of frames containing the same state and next-choice position. Recursion changes syntax, not the amount of unfinished work.

Run the companion

BASH

```
javac --release 21 -Xlint:all -Werror RecursionInterviewChecks.java
java RecursionInterviewChecks
```

Expected final line: `PASS 17 recursion checks`.

SDE-2 CORE CHAPTER

Chapter 6 - Converting Recursion to Iteration

Why this chapter exists

Java does not perform tail-call optimization. A recursive method that calls itself ten thousand deep will throw `StackOverflowError` regardless of how simple the call is, and no compiler flag changes that. This is not a Java quirk to complain about - it is a constraint you design around, and interviewers test whether you can.

The question arrives in two forms. **"What happens if the input is a million elements?"** is asking whether you know the stack will blow. **"Now do it iteratively"** is asking whether you can perform the conversion systematically rather than by inventing a new algorithm under pressure.

The conversion is mechanical once you know the recipe, and the recipe differs depending on where the recursive call sits.

Why Java has no tail-call optimization

A tail call is a recursive call in the final position, where nothing remains to be done after it returns:

JAVA

```java
// Tail recursive: nothing happens after the call.
static int sumTail(int[] values, int index, int accumulator) {
    if (index == values.length) {
        return accumulator;
    }
    return sumTail(values, index + 1, accumulator + values[index]);
}

// NOT tail recursive: the addition happens after the call returns.
static int sumNaive(int[] values, int index) {
    if (index == values.length) {
        return 0;
    }
    return values[index] + sumNaive(values, index + 1);
}
```

A language with TCO reuses the current stack frame for `sumTail`, making it a loop. Java does not, and both versions overflow at the same depth.

The reason is deliberate. The JVM's security and stack-inspection model - historically `SecurityManager`, and still stack traces and `StackWalker` - depends on frames being real. Eliminating them changes observable behaviour. The Loom and Valhalla work has revisited this, but as of Java 25 there is no TCO, and writing `return f(...)` gains nothing.

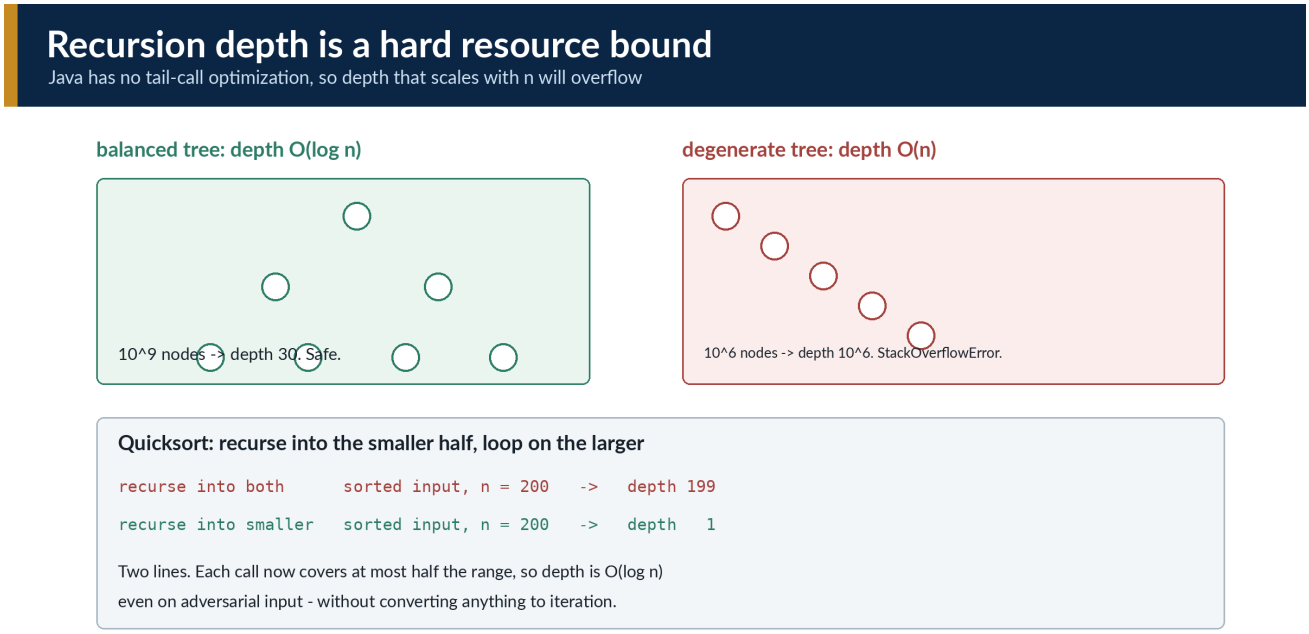
The practical consequence: the depth at which you overflow depends on frame size and the `-Xss` setting, typically somewhere in the range of a few thousand to a few tens of thousands of frames. Treat it as "roughly ten thousand" and never as a number to rely on.

> **Specification boundary:** the JVM specification does not mandate a stack size, does not require TCO, and does not define the depth at which `StackOverflowError` is thrown. It is a property of the implementation, the platform, the frame size, and `-Xss`. Any algorithm whose recursion depth scales with input size is therefore unbounded in a way the language will not protect you from.

Recognizing the danger before it fires

Recursion depth is safe when it is bounded by something small, and dangerous when it scales with `n`:

Shape	Depth	Safe?
Balanced tree traversal	O(log n)	Yes - 10^9 nodes is depth 30
Degenerate tree traversal	O(n)	**No** - a linked-list-shaped tree overflows
Linked-list recursion	O(n)	**No**
Quicksort, recursing into both halves	O(n) worst case	**No** - recurse into the smaller side only
Merge sort	O(log n)	Yes
DFS on a graph	O(V)	**No** on a long path
Backtracking - permutations, n-queens	O(n) where n is small	Usually yes



Java SDE-2 DSA Series | Volume 09: Recursion | Figure 01

Depth is safe when bounded by log n and dangerous when it scales with n

The tree row is the trap. A recursive traversal is perfectly safe on the balanced trees used in examples and overflows on the degenerate one an adversarial test supplies. "It works on my test data" is exactly the failure mode.

Backtracking is usually fine because depth is bounded by the *solution* size - permutations of 10 elements recurse 10 deep, not 10! deep. Knowing which dimension bounds the depth is the distinction.

Conversion recipe 1: tail position becomes a loop

When the recursive call is the last action and nothing is pending, replace the call with reassignment and loop.

JAVA

```java
// Recursive
static int gcd(int a, int b) {
    if (b == 0) {
        return a;
    }
    return gcd(b, a % b);
}

// Iterative: parameters become mutable locals, the call becomes a loop step
static int gcdIterative(int a, int b) {
    while (b != 0) {
        int remainder = a % b;
        a = b;
        b = remainder;
    }
    return a;
}
```

The mechanical rule: **each recursive call's arguments become assignments to the parameter variables, and the whole body becomes a `while` loop.** The base case becomes the loop's exit condition.

This is the easy case, and it is also the one people wrongly assume the JVM will handle for them.

Conversion recipe 2: one pending operation becomes an accumulator

If work remains after the recursive call, first try to move that work *before* the call by carrying an accumulator. This transforms non-tail recursion into tail recursion, which then becomes a loop by recipe 1.

JAVA

```java
// Non-tail: the multiply happens after the call
static long factorial(int n) {
    return n <= 1 ? 1 : n * factorial(n - 1);
}

// Accumulator-passing: now tail recursive
static long factorialTail(int n, long accumulator) {
    return n <= 1 ? accumulator : factorialTail(n - 1, n * accumulator);
}

// Loop
static long factorialIterative(int n) {
    long accumulator = 1;
    for (int i = 2; i <= n; i++) {
        accumulator *= i;
    }
    return accumulator;
}
```

This works when the pending operation is **associative** - addition, multiplication, min, max, string concatenation. It does not work when the pending work depends on the *result* in a way that cannot be reordered.

Reversing a linked list is the canonical case where accumulator passing is not just possible but produces the standard iterative solution:

JAVA

```java
static ListNode reverse(ListNode head) {
    ListNode previous = null;               // the accumulator
    ListNode current = head;
    while (current != null) {
        ListNode next = current.next;       // save before overwriting
        current.next = previous;            // the pending work, done early
        previous = current;
        current = next;
    }
    return previous;
}
```

Conversion recipe 3: two or more calls need an explicit stack

When a method recurses more than once - tree traversal, divide and conquer - no accumulator trick applies. You must simulate the call stack.

The mechanical translation: **push what the recursive calls would have been, in reverse order, so they pop in the original order.**

JAVA

```java
// Recursive preorder
static void preorder(TreeNode node, List<Integer> out) {
    if (node == null) {
        return;
    }
    out.add(node.val);
    preorder(node.left, out);
    preorder(node.right, out);
}

// Iterative: push right first so left pops first
static List<Integer> preorderIterative(TreeNode root) {
    List<Integer> out = new ArrayList<>();
    if (root == null) {
        return out;
    }
    Deque<TreeNode> stack = new ArrayDeque<>();
    stack.push(root);
    while (!stack.isEmpty()) {
        TreeNode node = stack.pop();
        out.add(node.val);
        if (node.right != null) {
            stack.push(node.right);      // pushed first, popped last
        }
        if (node.left != null) {
            stack.push(node.left);
        }
    }
    return out;
}
```

Preorder is the easy one because the visit happens *before* both calls. Inorder and postorder are harder, because the method must resume at a point *between* or *after* the recursive calls - and that resumption point is state the recursive version stored implicitly in the program counter.

The general technique: an explicit state machine

When a method has several resumption points, make the state explicit. Each stack frame records both the data and *where in the method to continue*.

JAVA

```
static List<Integer> postorderIterative(TreeNode root) {
    List<Integer> out = new ArrayList<>();
    if (root == null) {
        return out;
    }
    // phase 0 = descend left, 1 = descend right, 2 = visit
    record Frame(TreeNode node, int phase) {}
    Deque<Frame> stack = new ArrayDeque<>();
    stack.push(new Frame(root, 0));

    while (!stack.isEmpty()) {
        Frame frame = stack.pop();
        TreeNode node = frame.node();
        switch (frame.phase()) {
            case 0 -> {
                stack.push(new Frame(node, 1));            // resume here after left
                if (node.left != null) {
                    stack.push(new Frame(node.left, 0));
                }
            }
            case 1 -> {
                stack.push(new Frame(node, 2));            // resume here after right
                if (node.right != null) {
                    stack.push(new Frame(node.right, 0));
                }
            }
            default -> out.add(node.val);
        }
    }
    return out;
}
```

This is verbose, and that verbosity is the honest cost of the conversion. But it is **fully general**: any recursive method converts this way, with one phase per resumption point. When an interviewer asks for an iterative version of something awkward, this recipe always works, and saying "I would make the resumption point explicit" is a better answer than searching for a clever trick.

There is a well-known shortcut for postorder - do a reversed preorder visiting right before left, then reverse the output - and it is worth knowing. But it is specific to postorder, whereas the phase technique generalizes.

Bounding depth instead of converting

Sometimes the cheapest fix is not conversion but reducing the depth.

Recurse into the smaller half. Quicksort's worst-case O(n) depth becomes O(log n) if you always recurse into the smaller partition and loop on the larger:

JAVA

```java
static void quicksort(int[] values, int low, int high) {
    while (low < high) {
        int pivot = partition(values, low, high);
        if (pivot - low < high - pivot) {
            quicksort(values, low, pivot - 1);   // recurse into the smaller side
            low = pivot + 1;                     // loop on the larger
        } else {
            quicksort(values, pivot + 1, high);
            high = pivot - 1;
        }
    }
}
```

Depth is now O(log n) even on adversarial input, because each recursive call handles at most half the range. This is a two-line change with a large effect, and it is a good answer to "your quicksort overflows on sorted input".

Run on a dedicated thread with a larger stack when the recursion is genuinely deep and conversion is not worth it:

JAVA

```java
Thread worker = new Thread(null, task, "deep-recursion", 64 * 1024 * 1024);
```

The four-argument `Thread` constructor takes a stack size. This is a legitimate engineering answer for a batch job, and a poor one for a request handler - it raises the ceiling without removing it. Note also that **virtual threads do not accept a stack-size hint**; their stacks grow on the heap, which changes the failure mode but does not give unbounded depth.

WATCH OUT | Edge cases and common mistakes

- Assuming the JVM optimizes tail calls. It does not, at any release.
- Writing `return f(...)` believing it costs no frame.
- Testing recursion only on balanced trees and shipping it against degenerate ones.
- Recursing into both quicksort partitions, leaving O(n) worst-case depth.
- Pushing children in the wrong order and getting a mirrored traversal.
- Converting inorder or postorder by pattern-matching on preorder's shape without handling resumption.
- Catching `StackOverflowError` and continuing; the stack is in an unknown state and recovery is unreliable.
- Relying on a specific overflow depth. It varies with platform, frame size, and `-Xss`.
- Passing a stack size to a virtual thread, which ignores it.
- Converting to iteration when reducing depth would do, at a fraction of the complexity.
- Forgetting that an explicit stack still uses O(n) heap; conversion removes the *stack* limit, not the space cost.

TRY IT | Interview questions and model answers

Does Java optimize tail recursion?

No, at any release. Writing the call in tail position gains nothing, and the method overflows at the same depth as the non-tail version. The reason is that the JVM's stack-inspection behaviour - stack traces, `StackWalker`, historically the security manager - depends on frames actually existing.

Your recursive tree traversal is given a million nodes. What happens?

It depends on the shape. A balanced tree is depth 20 and fine. A degenerate tree - every node having one child - is depth one million and throws `StackOverflowError`. Since the shape is usually not guaranteed, I would convert to an explicit stack, or use Morris traversal if O(1) space is also wanted.

Convert this recursion to iteration.

It depends where the call sits. Tail position becomes a `while` loop with the arguments reassigned to the parameters. One pending associative operation becomes an accumulator carried forward, which makes it tail recursive first. Two or more calls need an explicit stack, pushing in reverse so they pop in order - and if the method resumes at several points, each stack frame carries a phase saying where to continue. That last recipe is fully general.

Your quicksort overflows on sorted input. Fix it without converting to iteration.

Recurse into the smaller partition and loop on the larger. Each recursive call then covers at most half the range, so depth is O(log n) even in the worst case. Two lines, and it removes the overflow entirely.

Can you just catch `StackOverflowError`?

You can catch it, but you should not treat it as recoverable. It is an `Error`, the stack is in an unknown state, and any cleanup you attempt may itself need frames you do not have. It is a signal that the design assumed bounded depth and the assumption was wrong.

When would you increase the stack size instead of converting?

For a batch or offline job where the depth is known and bounded and the conversion is not worth the complexity - a dedicated thread with a large stack via the four-argument `Thread` constructor. Not for a request handler, because it raises the ceiling rather than removing it, and every concurrent request pays the memory. Virtual threads ignore the hint entirely.

TRY IT | Exercises

1. **Foundation:** Write a recursive sum over an array and find, empirically, the depth at which it overflows on your JVM. Then change `-Xss` and measure again.
2. **Foundation:** Convert `gcd` and `factorial` to iteration using recipes 1 and 2. State which recipe each needed and why.
3. **Interview Core:** Convert recursive preorder, inorder, and postorder to iteration. Use the phase technique for postorder, then compare against the reverse-preorder shortcut.
4. **Interview Core:** Build a degenerate tree of 10^6 nodes and demonstrate the recursive traversal overflowing where the iterative one succeeds.
5. **Interview Core:** Implement quicksort recursing into both halves, overflow it on sorted input, then apply the smaller-half fix and show the depth drop.
6. **Interview Core:** Convert recursive linked-list reversal to iteration and identify what plays the role of the accumulator.
7. **SDE-2 Follow-up:** Take a recursive method with three resumption points and convert it with the phase technique. Count the phases before writing code.
8. **SDE-2 Follow-up:** Run a deep recursion on a `Thread` with a 64 MB stack, then on a virtual thread, and describe how the failure differs.
9. **SDE-2 Follow-up:** Convert recursive DFS on a graph to iteration and confirm the visit order matches.
10. **Challenge:** Write a general converter for a two-call recursive method: given the recursive form, produce the phase-based iterative version and verify equivalence on random inputs.

RECAP | Chapter summary

Java performs no tail-call optimization and never has, so recursion depth is a hard resource bound rather than a stylistic concern, and the depth at which it fails is an implementation property you cannot rely on. Depth is safe when bounded by something small - a balanced tree, a backtracking solution size - and dangerous whenever it scales with `n`, which is why a traversal that passes every balanced test overflows on a degenerate one. Conversion follows three recipes by call position: tail position becomes a loop with reassigned parameters; a single pending associative operation becomes an accumulator that makes the call tail recursive first; and two or more calls need an explicit stack, with a phase recorded per frame when the method resumes at more than one point. That last recipe is verbose and fully general, and naming it is a better answer than hunting for a trick. Before converting at all, consider bounding the depth instead - recursing into quicksort's smaller partition turns worst-case O(n) depth into O(log n) for two lines.

TRY IT | Revision checklist

- [ ] I know Java has no TCO and can say why.
- [ ] I can identify which recursions have depth proportional to input size.
- [ ] I know a degenerate tree breaks a traversal that balanced tests pass.
- [ ] I can convert tail recursion to a loop mechanically.
- [ ] I can introduce an accumulator and say when it is valid.
- [ ] I can convert multi-call recursion with an explicit stack, pushing in reverse.
- [ ] I can apply the phase technique when a method has several resumption points.
- [ ] I can fix quicksort's depth by recursing into the smaller partition.
- [ ] I know `StackOverflowError` is not meaningfully recoverable.
- [ ] I know virtual threads ignore a stack-size hint.

SDE-2 CORE CHAPTER

Chapter 7 - Recursion and Backtracking Practice Lab

Knowledge checks

1 **Foundation:** What must become smaller or closer to completion on every recursive branch?

2 **Foundation:** Why is a shared path list removed from after a child returns?

3 **Interview Core:** Distinguish pruning from memoization.

4 **SDE-2 Follow-up:** Why can a recursive O(n) algorithm still be unsafe for large input?

Predict and debug

5 Predict the output order of a preorder countdown that prints before recursing from 3 to 0.

6 What happens if a subsets method adds the mutable `path` reference to every leaf without copying?

7 Fix a permutation method that marks `used[i] = true` but never resets it.

8 A combination-sum solution prunes when the remaining target is negative, but values may be negative. Explain the correctness problem.

Coding tasks

9 Implement recursive exponentiation by squaring, including negative-exponent contract discussion.

10 Generate all balanced parentheses for `n` pairs with sound pruning.

11 Return only the count of N-Queens arrangements without materializing boards.

12 Solve a maze while restoring the input on every exit path.

13 Convert a deep recursive graph traversal to an explicit `ArrayDeque` stack.

Interview follow-ups

14 When would you use a boolean return instead of collecting all solutions?

15 Explain why output-sensitive complexity is more honest for subsets and permutations.

16 Describe how cancellation or time budgets should interact with restoration.

Essential clinic task

17 **SDE-2 Follow-up:** Implement a Sudoku solver that validates the initial board, uses row/column/box constraint state, restores every failed choice, and distinguishes invalid input from a valid board with no completion.

Recursion engine lab

18 **Foundation:** Write the exact enter/return frame trace for recursive sum over `[4,7,2]`.

19 **Interview Core:** Implement unique permutations for duplicate values and justify the sibling-skip condition using `used[]` state.

20 **Interview Core:** Implement reusable-candidate combination sum; reject any candidate that prevents strict progress.

21 **Interview Core:** Count N-Queens solutions using column and two diagonal arrays. Define the `n=0` result.

22 **Interview Core:** Implement word search without a board sentinel, support jagged rows, and prove state restoration after success and failure.

23 **SDE-2 Follow-up:** Differential-check unique permutation count against `n! / product(frequency!)` on small random multisets, then explain why this test is not a proof.

SDE-2 CORE CHAPTER

Chapter 8 - Recursion and Backtracking Practice Lab Solutions

1 A well-founded progress measure must move toward a base case: a smaller length, later index, lower remaining decision count, or structurally smaller subtree.

2 Sibling branches share the list. Removing restores the caller's invariant so one branch's choice does not leak into the next.

3 Pruning proves a branch cannot contain a required answer. Memoization caches the answer for a repeated state. Backtracking may have few repeated states even when its tree is large.

4 Java consumes one stack frame per active call and does not promise tail-call elimination. O(n) work with depth O(n) can overflow the stack.

5 It prints `3 2 1` when printing before the recursive call; printing after it would produce `1 2 3` during unwinding.

6 Every output entry aliases the same list and eventually reflects its final restored state, commonly appearing empty.

7 Reset `used[i] = false` after the recursive call, paired with removal of the chosen path value.

8 A negative remaining target can later rise when a negative candidate is chosen, so the prune can discard valid solutions. The problem also needs a termination rule to prevent unlimited reuse cycles.

9 For nonnegative exponent, return 1 at zero, recurse on `exponent / 2`, square once, and multiply by the base for odd exponents. For negative exponent, choose floating-point output and guard the minimum integer negation case with a wider type.

10 Track counts of open and close parentheses. Add an open parenthesis while `open < n`; add a close while `close < open`. A leaf occurs at length `2n`.

11 Track occupied columns and diagonals in boolean arrays or bit sets. Increment a counter at row `n`; do not build strings or boards.

12 Save the cell, mark it, explore, and restore before every return. `try/finally` is appropriate if deeper code can throw or observe cancellation.

13 Push the start, then pop, skip already visited nodes, mark, process, and push neighbors. To preserve recursive order, push neighbors in reverse order.

14 Use boolean short-circuiting for existence or first-solution queries; collect only when the contract requires enumeration.

15 The output itself can be exponential and includes copied elements. Complexity should include nodes visited and total emitted representation.

16 Check a cancellation token at defined boundaries and guarantee undo in `finally`. Partial output and input mutation policies must be documented.

17 Validate shape, symbols, and duplicate starting digits while initializing three `boolean[9][10]` tables. At an empty cell, try only digits unused by its row, column, and box; mark, recurse, and unmark on failure. Returning `false` after complete search means a valid initial board has no solution, while malformed or contradictory input should be rejected before search. Minimum-remaining-values ordering can reduce the practical tree without changing correctness.

Recursion engine solutions

18 Entry order is lengths 3, 2, 1, 0. Return events are `(0,0)`, `(1,4)`, `(2,11)`, `(3,13)`. Each suspended frame adds its own last included value only after the smaller call returns.

19 Sort a clone, track used indexes, and at each depth skip index `i` when it equals `i-1` and the earlier twin is not used in this branch. Mark, append, recurse, remove, unmark. Snapshot only at path length `n`. This permits both copies while preventing equivalent sibling orders.

20 Validate target nonnegative and every candidate positive, sort, and skip equal siblings. Recurse with the same index to permit reuse and subtract the chosen candidate. Positivity guarantees remaining target decreases and allows breaking when candidate exceeds it. Zero would create a nonprogressing branch.

21 One row is decided per depth. Reject a column when `column[c]`, `descending[row-c+n-1]`, or `ascending[row+c]` is set. Mark all, recurse, then unmark. When row equals `n`, return one arrangement; therefore `n=0` returns one empty arrangement.

22 Allocate a visited row matching every board row. Match bounds/current character, return at the final character, otherwise mark, explore four directions, and unmark before returning the combined result. A separate visited matrix avoids sentinel collisions; validation rejects null rows while per-row bounds support jagged input.

23 Generate small values from a tiny domain, ensure produced lists are unique, and compare count with `n! / product(frequency!)`. A fixed seed makes failures repeatable. It does not prove branch restoration or correctness for all sizes; targeted empty/all-equal cases and the canonical-sibling argument are still required.

SDE-2 CORE CHAPTER

Chapter 9 - Executable Recursion and Backtracking Checks

This dependency-free Java 21 companion validates frame contracts, restoration, duplicate-aware generation, combination search, N-Queens state, jagged-board word search, and differential oracles. A successful run prints PASS 17 recursion checks.

JAVA (continued 1/9)

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashSet;
import java.util.List;
import java.util.Random;
import java.util.Set;

public final class RecursionInterviewChecks {
    private RecursionInterviewChecks() {}

    static long sum(int[] values, int length) {
        if (length < 0 || length > values.length) {
            throw new IllegalArgumentException("invalid length");
        }
        return length == 0 ? 0L : sum(values, length - 1) + values[length - 1];
    }

    static List<String> sumFrameTrace(int[] values) {
        List<String> trace = new ArrayList<>();
        tracedSum(values, values.length, trace);
        return trace;
    }

    private static long tracedSum(int[] values, int length, List<String> trace) {
        trace.add("enter length=" + length);
        if (length == 0) {
            trace.add("return length=0 value=0");
            return 0;
        }
        long result = tracedSum(values, length - 1, trace) + values[length - 1];
        trace.add("return length=" + length + " value=" + result);
        return result;
    }

    static List<List<Integer>> uniqueSubsets(int[] input) {
        int[] values = input.clone();
```

JAVA (continued 2/9)

```
        Arrays.sort(values);
        List<List<Integer>> result = new ArrayList<>();
        build(values, 0, new ArrayList<>(), result);
        return result;
    }

    private static void build(int[] values, int start, List<Integer> path,
                              List<List<Integer>> result) {
        result.add(new ArrayList<>(path));
        for (int i = start; i < values.length; i++) {
            if (i > start && values[i] == values[i - 1]) {
                continue;
            }
            path.add(values[i]);
            build(values, i + 1, path, result);
            path.remove(path.size() - 1);
        }
    }

    static List<String> balancedParentheses(int pairs) {
        if (pairs < 0) {
            throw new IllegalArgumentException("negative pairs");
        }
        List<String> result = new ArrayList<>();
        buildParentheses(pairs, 0, 0, new StringBuilder(), result);
        return result;
    }

    private static void buildParentheses(int pairs, int open, int close,
                                         StringBuilder path, List<String> result) {
        if (path.length() == pairs * 2) {
            result.add(path.toString());
            return;
        }
        if (open < pairs) {
            path.append('(');
```

JAVA (continued 3/9)

```
            buildParentheses(pairs, open + 1, close, path, result);
            path.deleteCharAt(path.length() - 1);
        }
        if (close < open) {
            path.append(')');
            buildParentheses(pairs, open, close + 1, path, result);
            path.deleteCharAt(path.length() - 1);
        }
    }

    static List<List<Integer>> uniquePermutations(int[] input) {
        int[] values = input.clone();
        Arrays.sort(values);
        List<List<Integer>> result = new ArrayList<>();
        permute(values, new boolean[values.length], new ArrayList<>(), result);
        return result;
    }

    private static void permute(
            int[] values,
            boolean[] used,
            List<Integer> path,
            List<List<Integer>> result) {
        if (path.size() == values.length) {
            result.add(new ArrayList<>(path));
            return;
        }
        for (int index = 0; index < values.length; index++) {
            if (used[index]) {
                continue;
            }
            if (index > 0 && values[index] == values[index - 1] && !used[index - 1]) {
                continue;
            }
            used[index] = true;
            path.add(values[index]);
```

JAVA (continued 4/9)

```
            permute(values, used, path, result);
            path.remove(path.size() - 1);
            used[index] = false;
        }
    }

    static List<List<Integer>> combinationSum(int[] input, int target) {
        if (target < 0) {
            throw new IllegalArgumentException("target cannot be negative");
        }
        int[] candidates = input.clone();
        Arrays.sort(candidates);
        for (int candidate : candidates) {
            if (candidate <= 0) {
                throw new IllegalArgumentException("positive candidates required");
            }
        }
        List<List<Integer>> result = new ArrayList<>();
        combine(candidates, target, 0, new ArrayList<>(), result);
        return result;
    }

    private static void combine(
            int[] candidates,
            int remaining,
            int start,
            List<Integer> path,
            List<List<Integer>> result) {
        if (remaining == 0) {
            result.add(new ArrayList<>(path));
            return;
        }
        for (int index = start; index < candidates.length; index++) {
            if (index > start && candidates[index] == candidates[index - 1]) {
                continue;
            }
```

JAVA (continued 5/9)

```
            int candidate = candidates[index];
            if (candidate > remaining) {
                break;
            }
            path.add(candidate);
            combine(candidates, remaining - candidate, index, path, result);
            path.remove(path.size() - 1);
        }
    }

    static int countNQueens(int size) {
        if (size < 0 || size > 15) {
            throw new IllegalArgumentException("size must be in [0,15]");
        }
        return placeQueen(0, size, new boolean[size],
                new boolean[Math.max(0, size * 2 - 1)],
                new boolean[Math.max(0, size * 2 - 1)]);
    }

    private static int placeQueen(
            int row,
            int size,
            boolean[] columns,
            boolean[] descending,
            boolean[] ascending) {
        if (row == size) {
            return 1;
        }
        int solutions = 0;
        for (int column = 0; column < size; column++) {
            int downIndex = row - column + size - 1;
            int upIndex = row + column;
            if (columns[column] || descending[downIndex] || ascending[upIndex]) {
                continue;
            }
            columns[column] = true;
```

JAVA (continued 6/9)

```
            descending[downIndex] = true;
            ascending[upIndex] = true;
            solutions += placeQueen(row + 1, size, columns, descending, ascending);
            columns[column] = false;
            descending[downIndex] = false;
            ascending[upIndex] = false;
        }
        return solutions;
    }

    static boolean wordExists(char[][] board, String word) {
        if (board == null || word == null) {
            throw new IllegalArgumentException("board and word are required");
        }
        if (word.isEmpty()) {
            return true;
        }
        boolean[][] used = new boolean[board.length][];
        for (int row = 0; row < board.length; row++) {
            if (board[row] == null) {
                throw new IllegalArgumentException("null row");
            }
            used[row] = new boolean[board[row].length];
        }
        for (int row = 0; row < board.length; row++) {
            for (int column = 0; column < board[row].length; column++) {
                if (findWord(board, word, 0, row, column, used)) {
                    return true;
                }
            }
        }
        return false;
    }

    private static boolean findWord(
            char[][] board,
```

JAVA (continued 7/9)

```
            String word,
            int index,
            int row,
            int column,
            boolean[][] used) {
        if (row < 0 || row >= board.length || column < 0
                || column >= board[row].length || used[row][column]
                || board[row][column] != word.charAt(index)) {
            return false;
        }
        if (index + 1 == word.length()) {
            return true;
        }
        used[row][column] = true;
        boolean found = findWord(board, word, index + 1, row + 1, column, used)
                || findWord(board, word, index + 1, row - 1, column, used)
                || findWord(board, word, index + 1, row, column + 1, used)
                || findWord(board, word, index + 1, row, column - 1, used);
        used[row][column] = false;
        return found;
    }

    private static boolean permutationCountsMatchSetOracle() {
        Random random = new Random(47L);
        for (int trial = 0; trial < 200; trial++) {
            int[] values = new int[random.nextInt(8)];
            for (int index = 0; index < values.length; index++) {
                values[index] = random.nextInt(4);
            }
            List<List<Integer>> permutations = uniquePermutations(values);
            Set<List<Integer>> unique = new HashSet<>(permutations);
            if (unique.size() != permutations.size()) {
                return false;
            }
            long expected = factorial(values.length);
```

JAVA (continued 8/9)

```
            int[] frequencies = new int[4];
            for (int value : values) {
                frequencies[value]++;
            }
            for (int frequency : frequencies) {
                expected /= factorial(frequency);
            }
            if (permutations.size() != expected) {
                return false;
            }
        }
        return true;
    }

    private static long factorial(int value) {
        long result = 1;
        for (int factor = 2; factor <= value; factor++) {
            result *= factor;
        }
        return result;
    }

    private static void expectFailure(Runnable action) {
        try {
            action.run();
        } catch (IllegalArgumentException expected) {
            return;
        }
        throw new AssertionError("expected IllegalArgumentException");
    }

    private static void check(boolean condition, String message) {
        if (!condition) {
            throw new AssertionError(message);
        }
```

JAVA (continued 9/9)

```
    }

    public static void main(String[] args) {
        check(sum(new int[] {4, 7, 2}, 3) == 13L, "recursive sum");
        check(uniqueSubsets(new int[] {1, 2, 2}).size() == 6, "unique subsets");
        check(balancedParentheses(3).size() == 5, "parentheses");
        check(balancedParentheses(0).equals(List.of("")), "zero pairs");
        check(sumFrameTrace(new int[] {4, 7}).equals(List.of(
                "enter length=2", "enter length=1", "enter length=0",
                "return length=0 value=0", "return length=1 value=4",
                "return length=2 value=11")), "call-frame trace");
        expectFailure(() -> sum(new int[] {1}, 2));

        int[] permutationInput = {2, 1, 2};
        check(uniquePermutations(permutationInput).equals(List.of(
                List.of(1, 2, 2), List.of(2, 1, 2), List.of(2, 2, 1))),
                "unique permutations");
        check(Arrays.equals(permutationInput, new int[] {2, 1, 2}), "input preserved");
        check(combinationSum(new int[] {2, 3, 2, 7}, 7).equals(List.of(
                List.of(2, 2, 3), List.of(7))), "combination sum with duplicate candidates");
        expectFailure(() -> combinationSum(new int[] {0, 1}, 2));
        check(countNQueens(0) == 1, "empty board has one arrangement");
        check(countNQueens(4) == 2, "four queens");

        char[][] board = {{'A', 'B', 'C', 'E'}, {'S', 'F', 'C', 'S'}, {'A', 'D', 'E', 'E'}};
        check(wordExists(board, "ABCCED"), "word search true");
        check(!wordExists(board, "ABCB"), "cell cannot be reused");
        check(Arrays.deepEquals(board,
                new char[][] {{'A', 'B', 'C', 'E'}, {'S', 'F', 'C', 'S'}, {'A', 'D', 'E', 'E'}}),
                "word search preserves board");
        check(wordExists(new char[0][], ""), "empty word");
        check(permutationCountsMatchSetOracle(), "permutation count oracle");
        System.out.println("PASS 17 recursion checks");
    }
}
```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: factorial, tree height, and simple subsets
- Interview Core: permutations, combinations, and grid search
- SDE-2 Follow-up: bound depth and control allocation
- Challenge: design sound pruning rules

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: DSA 08 - Hashing: Maps, Sets, Frequency, and Prefix State

Series index

Next book: DSA 10 - Linked Lists: Pointer Reasoning and Mutation

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	**JAVA 01**	Java Foundations for Problem Solving
Java	**JAVA 02**	Git and GitHub for Java Engineers
Java	**JAVA 03**	Maven and Gradle for Java Engineers
Java	**JAVA 04**	Language, OOP, and Modern Java
Java	**JAVA 05**	Collections, Streams, and I/O
Java	**JAVA 06**	JVM and Java Execution
Java	**JAVA 07**	Java Concurrency and the Memory Model
Java	**JAVA 08**	Performance, Diagnostics, and GC Incidents
Java	**JAVA 09**	Java Question Bank, Study Plan, and Reference
DSA	**DSA 01**	Time and Space Complexity for Java Interviews
DSA	**DSA 02**	Number Systems and Math Foundations for DSA Interviews
DSA	**DSA 03**	Number Systems Interview Patterns and Rapid Revision
DSA	**DSA 04**	Bit Manipulation in Java
DSA	**DSA 05**	Loop Mastery, Patterns, and Index Calculations
DSA	**DSA 06**	Arrays and Array Problem-Solving Patterns
DSA	**DSA 07**	Strings and String Problem-Solving Patterns
DSA	**DSA 08**	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	**DSA 09**	Recursion and Backtracking in Java
DSA	**DSA 10**	Linked Lists: Pointer Reasoning and Mutation
DSA	**DSA 11**	Stacks, Queues, Deques, and Monotonic Patterns
DSA	**DSA 12**	Binary Search: Bounds, Answers, and Invariants
DSA	**DSA 13**	Trees, Binary Search Trees, and Tries
DSA	**DSA 14**	Heaps, Priority Queues, Selection, and Top-K
DSA	**DSA 15**	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	**DSA 16**	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	**DSA 17**	Dynamic Programming: State, Transitions, and Optimization
Frameworks	**FW 01**	MySQL for Java Backend Interviews
Frameworks	**FW 02**	Hibernate and JPA for Java Backend Interviews
Frameworks	**FW 03**	Spring Framework for Java Engineers
Frameworks	**FW 04**	Spring Boot for Java Backend Engineers
Frameworks	**FW 05**	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	**FW 06**	Spring Data for Java Backend Engineers
Frameworks	**FW 07**	MongoDB for Java Backend Interviews
Frameworks	**FW 08**	Redis for Java Backend Interviews
Frameworks	**FW 09**	Persistence, SQL, JPA, and Caching
Frameworks	**FW 10**	Apache Kafka and Spring Kafka
Frameworks	**FW 11**	Spring Ecosystem Extensions
Frameworks	**FW 12**	Spring AI for Java Engineers
System Design	**SD 01**	Design, Backend, Testing, and Security
System Design	**SD 02**	Distributed Systems and System Design
System Design	**SD 03**	Generative AI System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: https://github.com/vinayreddykalluri/SDE2-Interview-Handbook. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Data Structures and Algorithms - Book 09 of 17 - Study Step DSA-09. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.